

Hydra-NeXt: 基于开环训练的鲁棒闭环驾驶

Zhenxin Li^{1,2*} Shihao Wang^{3*} Shiyi Lan⁴ Zhiding Yu⁴ Zuxuan Wu^{1,2} Jose M. Alvarez⁴

¹Institute of Trustworthy Embodied AI, Fudan University

²Shanghai Collaborative Innovation Center of Intelligent Visual Computing

³The Hong Kong Polytechnic University ⁴NVIDIA

摘要

端到端自动驾驶研究目前面临一项关键挑战，即弥合开环训练与闭环部署之间的差距。当前方法在开环环境中训练以预测轨迹，但在闭环环境中难以对其他智能体做出快速反应，并且由于开环训练与闭环驾驶之间的差距，可能生成运动学上不可行的规划。本文提出 Hydra-NeXt，一种新颖的多分支规划框架，将轨迹预测、控制预测和轨迹细化网络统一于一个模型中。与仅处理一般情况规划的开环轨迹预测模型不同，Hydra-NeXt 进一步利用控制解码器专注于短期动作，从而能够更快地响应动态情况和反应性智能体。此外，本文提出轨迹细化模块，通过在闭环环境中有效遵循运动学约束来增强和细化规划决策。这种统一方法弥合了开环训练与闭环驾驶之间的差距，在 Bench2Drive 数据集上展示了 65.89 驾驶得分 (DS) 和 48.20% 成功率 (SR) 的优越性能，且无需依赖外部专家进行数据收集。Hydra-NeXt 比先前最先进的方法高出 22.98 DS 和 17.49 SR，标志着自动驾驶领域的重大进步。代码将在 <https://github.com/woxihuanjiangguo/Hydra-NeXt> 提供。

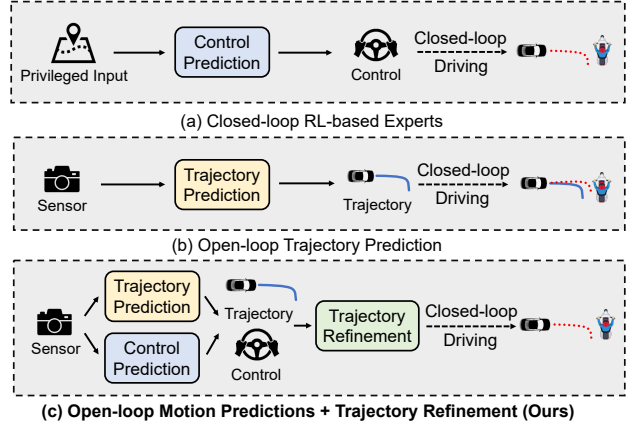


图 1. 自动驾驶的不同范式。基于闭环强化学习的专家从环境交互中学习，通过预测控制信号确保对智能体的响应性和运动学可行性。端到端方法使用开环轨迹预测，通常忽略自车与智能体的交互和运动学约束。本文方法整合了轨迹预测、控制预测和轨迹细化模块，以弥合开环训练与闭环驾驶之间的差距。

环境反馈。该方法的主要优势在于其简洁性与效率，因为它避免了实时反馈的复杂性，从而具有成本效益，并适用于大规模训练与测试。诸如导航模拟 (NAVSIM) [13] 等基准提供了受控环境，其中智能体遵循预定义轨迹，从而能够精确评估碰撞率、行驶进度及交通规则遵循等度量指标。然而，缺乏反应性行为以及与其他智能体或变化条件的动态交互，使得在开环环境中训练的模型难以部署于真实世界驾驶。自动驾驶中的闭环环境涉及智能体动作与环境之间的实时反馈，通常通过模拟器实现 [1, 4, 15]。闭环方法的主要优势在于其能够模拟动态、交互式环境，从而实现对驾驶策略更真实的测试。然而，它们面临显著挑战：这些系统通常依赖合成

1. 引言

端到端自动驾驶 (E2E AD) [7, 9, 21, 23, 25, 29, 33, 43, 48, 49, 51] 已成为传统感知-规划流程的替代趋势。当前 E2E AD 研究分为面向开环和闭环环境的方法。开环环境 [3, 4, 13, 23] 指自动驾驶系统的性能评估是在没有

* 在 NVIDIA 实习期间完成的工作。

模拟器数据，导致训练与真实世界部署之间存在领域差距。例如，先前最先进的端到端自动驾驶方法 [26] 严重依赖强化学习教师特征嵌入，而这些嵌入在真实世界数据中不可用。此外，学习到的驾驶行为可能偏离人类模式 [16, 32]，且评估指标主要关注碰撞率，忽视了平滑性和驾驶舒适性等其他重要因素 [28]。开环与闭环环境之间的差距导致各领域采用不同方法，且这种分离持续扩大。图 1 显示，闭环方法通过仿真中的强化学习学习控制 [26, 27, 32, 55]，而开环方法专注于模仿专家轨迹 [7, 23, 29]。两个领域之间的差距源于其训练方法和输出表示，这可以通过两种途径潜在解决。我们可以将在开环环境中训练的模型适配到闭环环境，或将闭环模型引入开放环境。我们认为第一种选择，即从开环到闭环，由于大量真实世界数据的可用性而更有前景。相比之下，第二种选择，即从闭环到开环，面临仿真到真实的领域差距 [20, 22, 31]，这构成了在真实车辆中部署模型的最大挑战。因此，我们专注于解决开环模型在闭环环境中驾驶时的局限性，这涉及处理反应性智能体并遵守运动学约束。作为 NAVSIM 基准 [13] 上最先进的开环端到端方法，Hydra-MDP 对非反应性智能体显示出有前景的结果，因此挑战在于将这种能力扩展到反应性智能体。非反应性智能体即使自车偏离真实轨迹也保持其行为。相比之下，反应性智能体会根据自车的预测轨迹进行调整，可能导致端到端自动驾驶模型的碰撞预测失效。开环训练引起的普遍问题是，大多数开环指标 [4, 13, 23] 关注航点而非控制信号。虽然航点对一般导航有用，但它们不考虑其他智能体行为的即时变化，导致意外事件发生时反应时间延迟。相比之下，控制信号直接影响车辆的动作，如制动或转向，从而能够更快地调整以应对突发变化。尽管现有文献 [51] 也直接预测控制与航点，但在交互场景中表现不佳 [28]，这可能源于缺乏多模态决策以及对其细微关系的探索不足。此外，预测的航点可能不满足闭环环境中的运动学约束，这可能导致复合误差 [12] 并在动态场景中产生不安全行为。因此，我们引入 Hydra-NeXt，具有多模态

分支规划框架。Hydra-NeXt 由用于轨迹和控制预测的多头运动解码器 (Multihead Motion Decoders) 以及用于基于运动学的候选方案选择的轨迹细化网络 (Trajectory Refinement network) 组成。除了用于处理一般情况规划的轨迹解码器 [33] 之外，控制解码器增强了短期动作的能力。此外，轨迹细化模块应用运动学约束来改进两个解码器的预测，有效地结合并细化规划决策。我们的贡献可总结如下：1. 我们提出了 Hydra-NeXt，这是第一个统一控制、轨迹预测和轨迹细化的框架。Hydra-NeXt 仅通过开环学习就展示了稳健的闭环驾驶性能。2. 我们在基于模拟器 CARLA v2 [1] 的 Bench2Drive 数据集 [28] 上，将 Hydra-NeXt 与其他端到端自动驾驶解决方案进行了基准测试。在不依赖外部专家收集数据的情况下，根据 CARLA v2 评估协议，Hydra-NeXt 以显著优势超越了先前最先进的方法（驾驶得分 +22.9，成功率 +17.5）。Hydra-NeXt 还很好地泛化到真实世界数据，在开环规划基准 NAVSIM [13] 上达到了新的最先进水平。

2. 相关工作

2.1. 开环端到端自动驾驶

为了利用真实世界车辆数据 [3] 探索端到端规划，研究人员已将模块化神经网络集成到端到端全可微堆栈 [21, 23, 29, 49] 中。这种方法迅速引起了自动驾驶领域的极大兴趣，因为它展示了一个有前景的可扩展自主方向。然而，最近的工作 [12, 13, 34] 发现当前端到端规划数据集 [13, 34]、模仿学习范式 [12, 34] 和评估协议 [13, 34] 存在严重偏差，导致开环端到端方法不可靠。为了解决这些问题，提出了更先进的基准 NAVSIM [13]，用于过滤同质化的规划数据，并使用一系列基于规则的度量进行基准测试，涵盖碰撞避免、地图遵循等。最先进的开环规划器 Hydra-MDP [33] 通过知识蒸馏学习这些基于规则的度量，实现了比基于模仿的端到端规划器更高的可靠性。在 Hydra-MDP 的基础上，我们开发了一个适用于动态场景的模型，从开环轨迹规划过渡到使用控制预测和基于运动学的轨迹细化的闭环框架。

2.2. 闭环驾驶基准与策略

与开环端到端自动驾驶相比，闭环驾驶的历史更为悠久。诸如 nuPlan [4]、CARLA v1 [15]、CARLA v2 [1] 和 Bench2Drive [28] 等基准侧重于驾驶策略的不同方面。

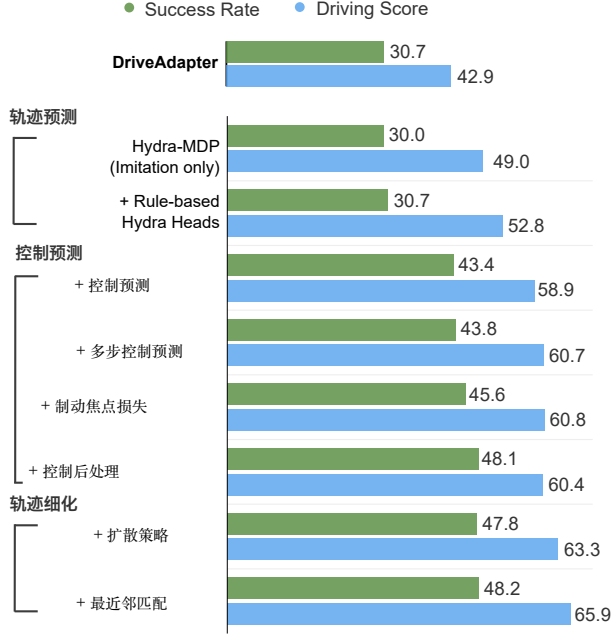


图 2. 从 Hydra-MDP 到 Hydra-NeXt 的路线图。DriveAdapter[26] 是 Bench2Drive 基准上先前最先进的方法。

从轨迹规划 [4] 到端到端驾驶 [1, 15, 28]。最近, CARLA v2[1] 提出了重大挑战, 因为它需要处理交互式动态场景。Bench2Drive[28] 优化了 CARLA v2 评估协议产生的方差, 并从基于强化学习的专家 [32] 在多样、较短的路线上收集数据, 使其适合在多变条件下评估策略。在这些基准上, 专家级策略包括基于规则的路径规划器 [2, 12] 和基于强化学习的策略 [16, 32, 55]。这些策略使用特权输入, 如真实感知和地图数据, 而端到端自动驾驶方法 [7, 9, 25, 43, 48, 51] 仅使用传感器观测。这些端到端策略以及先前的开环端到端方法在动态和交互场景中落后于专家级策略 [1]。此外, Bench2Drive 表明这些方法依赖专家特征嵌入 [26] 以获得更好的性能。在本文中, 我们旨在通过增强顶级开环规划器 [33] 的控制和运动学组件, 缩小开环规划与闭环驾驶之间的差距。

2.3. 基于扩散的驾驶策略

扩散策略 [6, 8, 52] 已被广泛用于生成机器人行为。它们被证明能有效捕捉多模态动作分布并生成平滑轨迹。在自动驾驶中, 近期工作利用扩散模型在给定周围交通场景的特权输入下预测轨迹航点 [17, 30, 50, 53]。我们的方法不同之处在于采用扩散策略生成

多个平滑且高频的控制序列, 作为轨迹细化的额外提议。

3. 轨迹解码器: Hydra-MDP

在介绍 Hydra-NeXt 之前, 我们从 Hydra-MDP [33] 开始, 这是一个多模态规划器, 包含感知网络和轨迹解码器, 从人类演示和基于规则的开放环度量中学习。

感知网络: 给定前视图和后视图图像, 我们使用图像骨干网络提取多视图特征。这些特征被展平为环境令牌序列 F_{env} 以表示周围环境。为了专注于运动规划, 我们在感知网络中采用极简设计, 不包含辅助感知任务 [7, 9, 23, 29, 49], 尽管它们可以促进规划。

轨迹解码器: 轨迹解码器 π_{traj} 基于环境令牌 F_{env} 和离散轨迹词汇表 $\mathcal{V}$ [7, 39, 40] 生成用于路由的轨迹 T 。遵循 Hydra-MDP [33], 我们创建了一个包含 4096 条离散轨迹的轨迹词汇表 $\mathcal{V}$, 将它们嵌入为潜在查询 Q_{traj} , 并在变换器解码器 [47] 中关注环境令牌 F_{env} 。仅通过模仿学习, 该方法在 Bench2Drive 上达到了 49.0 DS, 如图 2 所示, 比之前最先进的 DriveAdapter 高出 6.1 DS。此外, 我们在 Bench2Drive 上实现了开放环度量系统, 使用以下度量: • **碰撞:** 碰撞度量检查自车是否在鸟瞰图 (BEV) 空间中与其他智能体相交 [13, 23, 29, 34]。我们按照 NAVSIM [13] 通过插值轨迹航点以更高频率 (即 10Hz) 计算碰撞。

- **软车道保持:** 为了在允许变道的同时确保车道遵守, 我们设计了一个软车道保持度量, 限制自车的驾驶区域, 遵循 [29]。我们惩罚轨迹段与车道段之间角度差异过大的轨迹。

- **自车进度:** 引入自我进度度量是为了抑制被动驾驶行为并促进驾驶进度。与 [4, 13] 类似, 我们将轨迹航点投影到专家行驶经过的车道段上, 并用专家行驶距离对自我距离进行归一化。

这些度量在 Hydra-MDP 中用作基于规则的教师。图 2 展示了基于规则的头部带来的性能提升, 与在 NAVSIM[13] 上的观察相比, 这种提升似乎有限。这可能是由于 Bench2Drive 中随机消失的智能体使基于规则的头部产生混淆。例如, 一条延伸到很远未来的轨迹, 如果其未来航点不与已消失的智能体相交, 即使该智能体当前可见且靠近自我车辆, 也可能被标记为安全。在第 4 节中, 我们

¹<https://github.com/Thinklab-SJTU/Bench2Drive/issues/31>

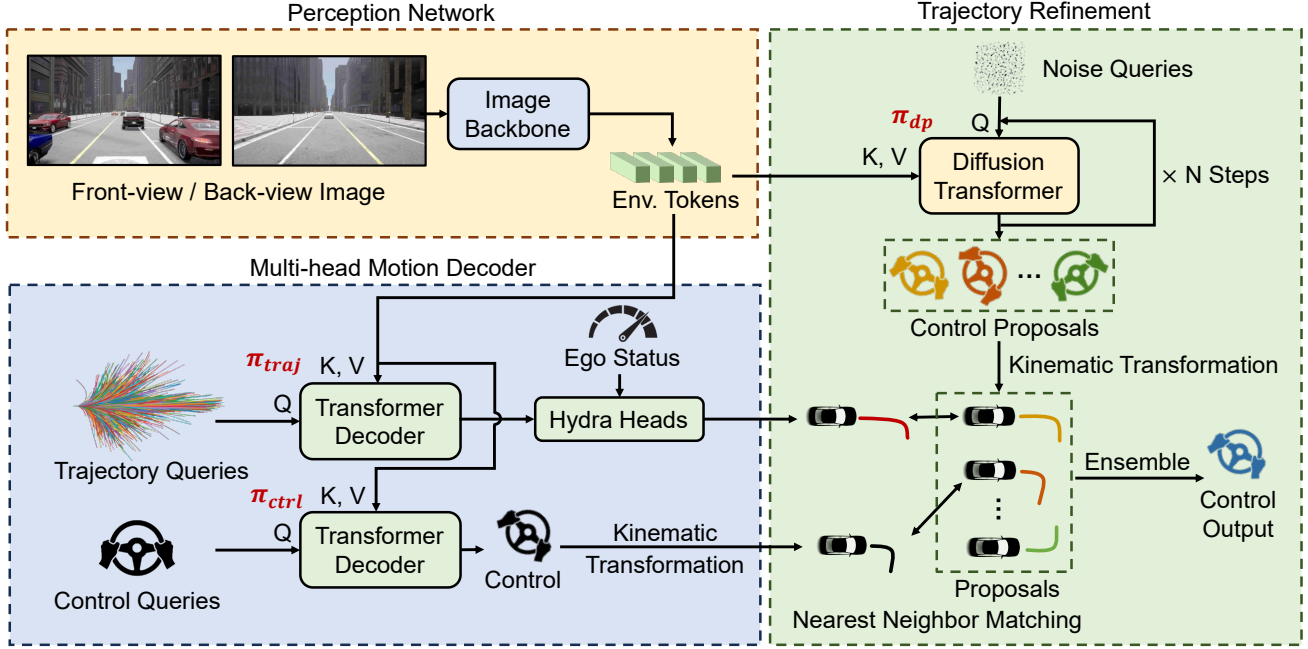


图 3. Hydra-NeXt 的整体架构。感知网络首先将原始传感器观测处理为环境令牌。多头运动解码器（轨迹解码器 π_{traj} 和控制解码器 π_{ctrl} ）负责基于这些令牌生成轨迹和控制元组，这些结果由轨迹细化模块 π_{dp} 细化为最终控制信号。

提出 Hydra-NeXt，这是 Hydra-MDP 的扩展版本，旨在解决闭环驾驶中的局限性。

4. Hydra-NeXt

在本节中，我们详细阐述 Hydra-NeXt，这是一个具有鲁棒闭环能力的端到端框架。除了轨迹解码器 π_{traj} 之外，Hydra-NeXt 还有另外两种策略：

控制解码器，以 π_{ctrl} 及轨迹细化 π_{dp} 。

4.1. 控制解码器 π_{ctrl}

我们的目标是通过直接控制输出实现对反应式智能体的快速响应，而非依赖轨迹航点进行快速决策。这是因为航点通常通过特定控制器（如比例-积分-微分控制器或模型预测控制 [5]）转换为控制信号，这可能在路径跟踪过程中引入误差。此外，TCP[51] 指出，将轨迹与控制信号集成可以提升驾驶性能，但其轨迹和控制预测分支均仅考虑单一模式。这一局限性使其难以应对环境中的不确定性 [7]，且容易在不同动作模式之间进行插值 [25]。基于这些发现，我们增加了一个基于分类的第二控制解码器 π_{ctrl} ，用于生成 t_{ctrl} 个时间步的控制信号 $(C^1, \dots, C^{t_{ctrl}})$ 。在 CALRA 模拟器 [1, 15] 中，每个控制信号是一个由 $(brake, throttle, steer)$ 组成的元组，用于引导车辆。同时，我们将离散化的思想融入到控制中。

控制解码器 π_{ctrl} 以处理不确定性，遵循基于强化学习的驾驶策略 Think2Drive[32] 的做法。采用基于学习的控制预测，避免了通过传统控制器将轨迹转换为控制信号的需要，从而能够在闭环环境中更快地响应反应式智能体。具体而言，本文首先随机初始化 t_{ctrl} 控制查询 Q_{ctrl} ，这些查询对应于我们希望预测控制信号的 t_{ctrl} 当前及未来时间步。类似地， Q_{ctrl} 在变换器解码器中关注 F_{env} 。此后， Q_{ctrl} 由三个独立的多层感知机层分别处理每个控制信号（i.e. $brake, throttle, steer$ ）。由于 Think2Drive 收集的专家数据仅包含离散控制信号，控制解码器可以直接使用交叉熵损失函数进行训练，而无需进一步离散化控制信号的真值。本文在每个时间步选择似然最高的控制元组 $(C^1, \dots, C^{t_{ctrl}})$ 作为 π_{ctrl} 的最终输出。控制解码器的设计选择（如架构和损失函数）将在第 5.4 节中讨论。

4.2. 轨迹细化 π_{dp} 闭环驾驶的另一个重要方面是确保驾驶的平顺性。诸如 nuPlan[4] 和 NAVSIM[13] 等基准利用线性二次调节器跟踪器 [46] 和运动学自行车模型 [41, 42] 对轨迹进行后处理，以符合运动学约束。然而，这一思想被现有的开环端到端自动驾驶方法所忽视，这些方法主要专注于克隆轨迹航点，而未考虑

在闭环环境中考虑运动学约束。当车辆在闭环环境中失去控制 [1, 28]，或在难以快速恢复到预测航路点时遇到较大的复合误差 [12]，这一疏忽就变得尤为关键。为解决这一问题，我们引入了轨迹细化网络，在遵循运动学约束的同时增强并组合规划输出 T 和 C 。受 PDM-Lite[2] 启发，我们采用运动学自行车模型 [41, 42]，该模型作用于高频控制信号，用于细粒度航路点展开 [2]。与 π_{traj} 预测的航路点不同，这些航路点在运动学上是可行的。然而，我们通过实验发现，现有控制策略难以在高频率下生成平滑而平滑的控制序列（见第 5.4 节）。因此，我们采用扩散策略 [8]，该策略被证明能够生成平滑且多样的机器人行为。在训练过程中，按照去噪扩散概率模型（DDPM）[19] 的常规做法，训练一个扩散 Transformer π_{dp} 来预测添加到真实控制序列上的噪声。在推理过程中， π_{dp} 通过多次去噪迭代，逐步将 N 随机噪声查询 Q_{dp} 去噪为平滑动作序列 $\{(\tilde{C}_i^1, \dots, \tilde{C}_i^{t_{dp}})\}_{i=1}^N$ 。每个提议跨越 t_{dp} 个时间步。利用这些平滑的控制序列提议，我们设计了一种名为最近邻匹配的过程，以运动学可行的方式选择最佳提议。算法 1 详细描述了用于提议选择的最近邻匹配过程。具体而言，我们首先使用运动学自行车模型将来自 π_{dp} 和 π_{ctrl} 的控制序列转换为轨迹航路点。转换后，我们基于 L2 距离选择两个最近的控制候选来匹配当前候选。新的控制候选可视为多头运动解码器预测的运动学可行版本。最后，我们通过平均油门和转向值将候选 C 集成为最终控制信号 C^* ，这是 TCP[51] 中一种简化的集成方法。如果条件 $\sum_{C \in \mathcal{C}} C.brake \geq \tau$ 成立，则制动设为 1；否则设为 0。注意，候选产生的制动值是二值的， τ 是预定义的制动阈值。

5. 实验

5.1. 数据集与度量

本文采用端到端驾驶基准 Bench2Drive [28] 对 Hydra-NeXt 进行训练与评估。本文训练数据使用 Bench2Drive 官方训练数据集，包含 200 万帧，涵盖 44 个交互场景。这些数据由 Think2Drive [32] 采集，该模型是基于 CARLA v2 [1] 的强化学习专家模型。评估时，端到端自动驾驶模型部署于 CARLA 模拟器 [15] 中，在 Bench2Drive 设计的 220 条短路径上执行闭环驾驶。

Algorithm 1: Nearest Neighbor Matching

Input: trajectory T , control sequence $(C^1, \dots, C^{t_{ctrl}})$, N control proposals $\{(\tilde{C}_i^1, \dots, \tilde{C}_i^{t_{dp}})\}_{i=1}^N$

Output: control candidates $\mathcal{C}$

Definitions : KB: Kinematic Bicycle Model

PID: PID controller

L₂: L₂ Distance

```

1  $\mathcal{C} \leftarrow \{\text{PID}(T), C^1\}$ 
2  $T_{ctrl} \leftarrow \text{KB}(C^1, \dots, C^{t_{ctrl}})$ 
3 for  $i \leftarrow 1$  to  $N$  do
4    $T_{dp}^i \leftarrow \text{KB}(\tilde{C}_i^1, \dots, \tilde{C}_i^{t_{dp}})$ 
5 end
6  $i \leftarrow \arg \min_i L_2(T_{dp}^i, T)$ 
7  $j \leftarrow \arg \min_j L_2(T_{dp}^j, T_{ctrl})$ 
8  $\mathcal{C} \leftarrow \mathcal{C} \cup \{\tilde{C}_i^1, \tilde{C}_j^1\}$ 

```

这些短路径用于评估自动驾驶模型在不同场景下的能力，同时确保最终得分的低方差。Bench2Drive 包含多项度量，包括驾驶得分（Driving Score, DS）、成功率（Success Rate, SR）、效率（Efficiency）、舒适度（Comfort）以及多能力结果。DS 的计算可基于两种评估协议：CARLA v2 与 Bench2Drive。前者将最低速度违规累积计入 DS，而 Bench2Drive 协议将其分离为效率度量。同时，后者协议放宽了闭环评估的时间约束，降低了模型完成路径的难度。若无特别说明，消融研究中默认采用 CARLA v2 评估协议，以 DS 度量反映方法的综合性能。此外，本文在真实世界 NAVSIM 基准 [13] 上评估 Hydra-NeXt，该基准使用开环度量评估 4 秒轨迹：无责任碰撞（No at-fault Collisions, NC）、可行驶区域合规性（Drivable Area Compliance, DAC）、碰撞时间（Time-to-collision, TTC）、舒适度（Comfort, C）以及自车进度（Ego Progress, EP）。PDM 得分（PDMS）是这些子度量的聚合。

5.2. 实现细节

Hydra-NeXt 的实现与 Bench2Drive 上的开环端到端基线 [23, 29] 基本一致。首先，本文在训练数据上训练 Hydra-NeXt 20 个训练轮次，总批大小为 256，使用 8 块 NVIDIA V100 GPU。采用 AdamW [37] 优化器与余弦退火调度器 [38]，学习率为 2×10^{-4} ，权重衰减为 0.01。训练期间，对输入图像应用随机裁剪和光度失真等数据增强，图像首先调整尺寸至 800×450 。

Method	Sensors	Open-loop Metric	Closed-loop Metrics (CARLA v2)	
		L2 (meter) ↓	Driving Score ↑	Success Rate (%) ↑
UniAD-Base [23]	6 Cameras	0.73	37.72	9.54
VAD [29]	6 Cameras	0.91	39.42	10.00
Hydra-MDP [33]	Front/Back Cameras	0.82	52.80	30.73
TCP* [51]	Front Camera	1.70	23.63	7.72
TCP-Ctrl* [51]	Front Camera	-	18.63	5.45
TCP-Traj* [51]	Front Camera	1.70	36.78	26.82
ThinkTwice* [27]	6 Cameras	0.95	39.88	28.14
DriveAdapter* [26]	6 Cameras	1.01	42.91	30.71
Hydra-NeXt	Front/Back Cameras	0.92	65.89	48.20

表 1. 端到端自动驾驶方法在 Bench2Drive 基准上使用 CARLA v2 评估协议的开环与闭环性能。CARLA v2 协议通过将所有违规（包括最低速度违规）相乘聚合来计算驾驶得分（DS）。* 该模型受益于专家特征蒸馏。

Method	Sensors	Closed-loop Metrics (Bench2Drive)			
		Driving Score ↑	Success Rate (%) ↑	Efficiency ↑	Comfort ↑
UniAD-Base [23]	6 Cameras	45.81	16.36	129.21	43.58
VAD [29]	6 Cameras	42.35	15.00	157.94	46.01
Hydra-MDP [33]	Front/Back Cameras	59.95	29.82	186.83	18.62
TCP* [51]	Front Camera	40.70	15.00	54.26	47.80
TCP-Ctrl* [51]	Front Camera	30.47	7.27	55.97	51.51
TCP-Traj* [51]	Front Camera	59.90	30.00	76.54	18.08
ThinkTwice* [27]	6 Cameras	62.44	31.23	69.33	16.22
DriveAdapter* [26]	6 Cameras	64.22	33.08	70.22	16.01
Hydra-NeXt	Front/Back Cameras	73.86	50.00	197.76	20.68

表 2. 端到端自动驾驶方法在 Bench2Drive 基准上的闭环性能，采用 Bench2Drive 评估协议。* 该模型受益于专家特征蒸馏。

Hydra-NeXt 采用 ImageNet 预训练的 [14]ResNet-50[18] 作为图像骨干网络，以提取前视图和后视图图像特征。遵循 VAD[29]，轨迹解码器 π_{traj} 以 2Hz 频率预测 3 秒轨迹。 π_{ctrl} 和 π_{dp} 的频率默认分别设为 2Hz 和 10Hz，将在第 5.4 节进一步分析。自车状态特征包括当前纵向速度和独热导航指令。在轨迹细化中，刹车阈值 τ 设为候选集大小 2 的一半以实现适度行为，而运动学自行车模型基于 PDM-Lite[2] 的实现， π_{dp} 的去噪时间步设为 100[8]。对于 NAVSIM，我们扩展 Hydra-MDP[33]，加入加速度和转向率预测作为控制信号的替代。详情见附录。

5.3. 主要结果

如表 1 和表 2 所示，Hydra-NeXt 在 Bench2Drive 上显著超越了所有端到端方法，尤其在驾驶得分和成功率等关键指标上表现突出。具体而言，在 CARLA v2 协议下，Hydra-NeXt 比之前最先进的 DriveAdapter [26] 高出 22.98 驾驶得分和 17.49% 成功率。在 Bench2Drive 协议下，它实现了 9.64 的提升

DS 和 16.92 SR，尽管 DriveAdapter 利用了来自 Think2Drive [32] 的专家特征。此外，Hydra-NeXt 展现出比基线更高的效率，尤其是与使用专家特征的基线相比。然而，Hydra-NeXt 在舒适性方面落后于 TCP [51]，这可能是由于交互场景中频繁制动所致。

对于表 3 所示的多能力结果，Hydra-NeXt 在平均性能上比 DriveAdapter 提升了 11.14%。值得注意的是，Hydra-NeXt 在交互场景中表现出优越的性能，例如并道（+11.18%）、超车（+38.06%）和紧急制动（+12.91%），这表明其擅长处理反应式智能体。然而，在车辆应遵守交通标志的场景中，Hydra-NeXt 落后于 DriveAdapter。这一现象可能是由于在交通标志前遇到混合专家行为时产生的因果混淆 [26] 所致，值得未来研究。值得注意的是，所有方法在让行特殊车辆（如救护车）方面均未取得令人满意的结果，这可能是由于此类训练数据稀缺所致。

此外，表 4 显示 Hydra-NeXt 在 NAVSIM 基准上达到了新的最先进水平。尽管 NAVSIM 仅评估来自 π_{traj} 的预测轨迹，

Method	Ability (%) $\uparrow$					
	Merging	Overtaking	Emergency Brake	Give Way	Traffic Sign	Mean
UniAD-Base [23]	14.10	17.78	21.67	10.00	14.21	15.55
VAD [29]	8.11	24.44	18.64	20.00	19.15	18.07
Hydra-MDP [33]	19.23	20.00	45.00	50.00	41.58	35.16
TCP* [51]	16.18	20.00	20.00	10.00	6.99	14.63
TCP-Ctrl* [51]	10.29	4.44	10.00	10.00	6.45	8.23
TCP-Traj* [51]	8.89	24.29	51.67	40.00	46.28	34.22
ThinkTwice* [27]	27.38	18.42	35.82	50.00	54.23	37.17
DriveAdapter* [26]	28.82	26.38	48.76	50.00	57.21	42.08
Hydra-NeXt	40.00	64.44	61.67	50.00	50.00	53.22

表 3. 端到端自动驾驶方法在 Bench2Drive 基准上的多能力结果。*表示专家特征蒸馏。

Method	NC $\uparrow$	DAC $\uparrow$	TTC $\uparrow$	EP $\uparrow$	C $\uparrow$	PDMS $\uparrow$
Transfuser [9]	97.7	92.8	92.8	79.2	100	84.0
Hydra-MDP [33]	98.3	96.0	94.6	78.7	100	86.5
DiffusionDrive [35]	98.2	96.2	94.7	82.2	100	88.1
Hydra-NeXt	98.1	97.7	94.6	81.8	100	88.6

表 4. 端到端自动驾驶方法在 NAVSIM 上的性能。

在 π_{ctrl} 和 π_{dp} 中扩展学习目标有助于轨迹预测，相比 Hydra-MDP[33] 提升了 2.1 PDMS。

5.4. 消融实验

不同组件及其设计的消融。图 2 展示了 Hydra-NeXt 中各组件的详细消融结果。在透视图像令牌上运行时，Hydra-MDP 达到了 52.8 的驾驶得分，已超越 DriveAdapter 约 10 个驾驶得分。然而，面对更复杂的智能体交互时，轨迹预测仍显不足，这促使我们利用控制预测网络 π_{ctrl} 实现快速反应。 π_{ctrl} 的输出以与第 4.2 节类似的方式与轨迹进行集成，其中在给定两个提议的情况下，制动阈值相应设为 1。这使成功率大幅提升了 12.7%，表明与反应型智能体的碰撞减少。此外，如 [51] 所示，将多步控制预测用于未来帧可提供辅助监督（+1.8 驾驶得分）。对制动预测应用二元焦点损失 [36] 可平衡训练（+1.8% 成功率），而对最终控制输出进行简单后处理（例如转弯时减速）可带来进一步提升（+2.5% 成功率）。最后，我们在先前网络之上构建了轨迹细化网络。引入扩散策略并进行直接集成可使驾驶得分提升约 3 分，而应用最近邻匹配（算法 1）以遵循运动学约束则将最终性能提升至 65.9 驾驶得分和 48.2% 成功率。

轨迹细化设计。为说明使用扩散策略 π_{dp} 作为提议生成器的必要性，我们首先将 π_{dp} 替换为离散控制解码器 π'_{ctrl}

Policy in Traj. Refine.	Driving Score $\uparrow$	Success Rate (%) $\uparrow$
π'_{ctrl}	60.15	42.25
π_{dp}	65.89	48.20

表 5. 轨迹细化中策略的消融。

π_{ctrl}	π_{dp}	Driving Score $\uparrow$	Success Rate (%) $\uparrow$
2 Hz	2 Hz	61.95	39.05
2 Hz	10 Hz	65.89	48.20
10 Hz	10 Hz	64.66	44.45

表 6. 控制解码器与扩散策略预测频率的消融。

Proposal Number N	Driving Score $\uparrow$	Success Rate (%) $\uparrow$
5	65.52	46.36
10	65.89	48.20
20	66.09	46.08

表 7. 轨迹细化中提议数量 N 的消融。

其架构与 π_{ctrl} 相似，同样可根据前 N 个置信度得分生成 N 个提议。如表 5 所示，这导致驾驶得分下降 5.78，可能源于 π'_{ctrl} 生成平滑控制序列的能力不足。此外，我们研究了 π_{dp} 对高频控制序列的建模能力。

表 6 表明，使用低频 π_{dp} 时性能有所下降。相反，低频 π_{ctrl} 优于高频。这与之前的发现相呼应，并证明了采用 π_{dp} 作为生成器的有效性。最后，表 7 考察了改变提议数量的影响。将提议数量 N 从 5 增加到 20，在 DS 上性能趋于饱和，而 SR 在较大的 N 下略有波动。这种波动可能源于 π_{dp} 与其他解码器之间的过度自信匹配，而我们可以潜在地从它们预测之间的微小差异中获益以进行探索。

5.5. 运行时效率与可视化

运行时效率。表 8 比较了 Hydra-NeXt 与端到端基线的运行时效率。由于 π_{traj} 执行

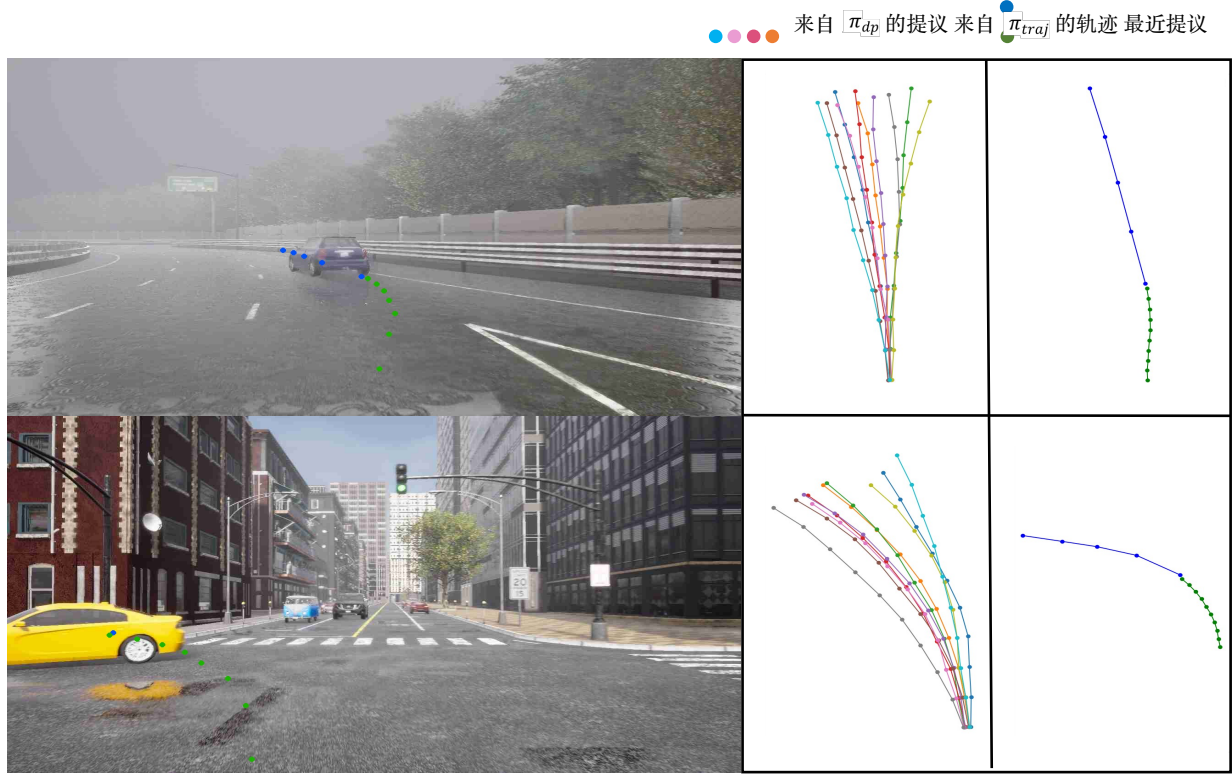


图 4. 轨迹细化可视化。该图展示了前视图图像、由控制提议变换得到的轨迹，以及所选的最接近预测轨迹的提议。

Method					Latency (ms)
UniAD-Base [23]					555.6
VAD [29]					224.3
DriveAdapter [26]					894.0
Method	π_{traj}	π_{ctrl}	π_{dp}	$\mathcal{F}$	Latency (ms)
Hydra-NeXt	✓				245.4
	✓	✓			250.6
	✓	✓	✓		998.6
	✓	✓	✓	✓	528.3

表 8. 运行时效率分析。Hydra-NeXt 和 VAD 的延迟在 NVIDIA RTX 3090 上进行基准测试，而 UniAD 和 DriveAdapter 分别在 NVIDIA Tesla A100 和 A6000 上进行。 $\mathcal{F}$ 指用于加速的 Flash 注意力 [11]。

基于透视图像标记进行规划，无需构建显式的鸟瞰图特征 [23]，因此比之前的 UniAD 实现了更高的效率。控制解码器 π_{ctrl} 也很轻量。然而，引入扩散策略 π_{dp} 来细化轨迹会导致效率下降，这是迭代去噪过程 [8] 的固有局限性，值得进一步优化。

可视化。图 4 展示了在高速公路合流场景和无保护左转场景中的轨迹细化过程。 π_{dp} 提出多条控制序列，并将其转换为轨迹。最佳匹配的

该方案能够平滑 π_{traj} 产生的大曲率轨迹，确保运动学可行性。

6. 结论

本文首先分析了当前端到端自动驾驶系统的发展，重点关注开环与闭环环境之间的差异，以及将开环训练模型应用于闭环驾驶所面临的挑战。这些挑战包括处理反应式智能体以及遵守运动学约束。为解决这些问题，本文提出 Hydra-NeXt，一种统一的轨迹与控制预测方法。通过将 Hydra-MDP 与多头运动解码器（Multi-head Motion Decoder）和轨迹细化网络（Trajectory Refinement network）相结合，Hydra-NeXt 获得了更强的闭环驾驶性能。具体而言，Hydra-NeXt 在闭环驾驶基准 Bench2Drive 上展现出卓越的性能，驾驶得分（DS）为 65.89，成功率（SR）为 48.20%，显著超越了以往最先进的方法。Hydra-NeXt 还在真实世界端到端规划基准 NAVSIM 上取得了新的最先进成果。致谢 本工作部分由国家自然科学基金（批准号 62472098）资助。本工作由上海市科学技术委员会（编号 24511103100）资助。

参考文献

- [1] Carla: Carla autonomous driving leaderboard (2022). <https://leaderboard.carla.org/>, 2022. 1, 2, 3, 4, 5, 12
- [2] Jens Beißwenger. PDM-Lite: A rule-based planner for carla leaderboard 2.0. <https://github.com/OpenDriveLab/DriveLM/blob/DriveLM-CARLA/docs/report.pdf>, 2024. 3, 5, 6, 11, 12
- [3] Holger Caesar, Varun Bankiti, Alex H Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuscenes: A multimodal dataset for autonomous driving. In *CVPR*, pages 11621–11631, 2020. 1, 2
- [4] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810*, 2021. 1, 2, 3, 4
- [5] Eduardo F. Camacho and Carlos Bordons Alba. *Model Predictive Control*. Springer Science & Business Media, 2013. 4
- [6] Minshuo Chen, Song Mei, Jianqing Fan, and Mengdi Wang. An overview of diffusion models: Applications, guided generation, statistical rates and optimization. *arXiv preprint arXiv:2404.07771*, 2024. 3
- [7] Shaoyu Chen, Bo Jiang, Hao Gao, Bencheng Liao, Qing Xu, Qian Zhang, Chang Huang, Wenyu Liu, and Xinggang Wang. Vadv2: End-to-end vectorized autonomous driving via probabilistic planning. *arXiv preprint arXiv:2402.13243*, 2024. 1, 2, 3, 4
- [8] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023. 3, 5, 6, 8, 11
- [9] Kashyap Chitta, Aditya Prakash, Bernhard Jaeger, Zehao Yu, Katrin Renz, and Andreas Geiger. Transfuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE TPAMI*, 2022. 1, 3, 7, 12
- [10] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024. 12
- [11] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 8, 12
- [12] Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *CoRL*, pages 1268–1281. PMLR, 2023. 2, 3, 5
- [13] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *NeurIPS*, 2024. 1, 2, 3, 4, 5, 12
- [14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, pages 248–255. Ieee, 2009. 6
- [15] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. Carla: An open urban driving simulator. In *CoRL*, pages 1–16. PMLR, 2017. 1, 2, 3, 4, 5, 12
- [16] Dechen Gao, Shuangyu Cai, Hanchu Zhou, Hang Wang, Iman Soltani, and Junshan Zhang. Cardreamer: Open-source learning platform for world model based autonomous driving. *arXiv preprint arXiv:2405.09111*, 2024. 2, 3
- [17] Zhiming Guo, Xing Gao, Jianlan Zhou, Xinyu Cai, and Botian Shi. Scenedm: Scene-level multi-agent trajectory generation with consistent diffusion models. *arXiv preprint arXiv:2311.15736*, 2023. 3
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, pages 770–778, 2016. 6, 12
- [19] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 33:6840–6851, 2020. 5
- [20] Sebastian Höfer, Kostas Bekris, Ankur Handa, Juan Camilo Gamboa, Melissa Mozifian, Florian Golemo, Chris Atkeson, Dieter Fox, Ken Goldberg, John Leonard, et al. Sim2real in robotics and automation: Applications and challenges. *IEEE transactions on automation science and engineering*, 18(2): 398–400, 2021. 2
- [21] Shengchao Hu, Li Chen, Penghao Wu, Hongyang Li, Junchi Yan, and Dacheng Tao. St-p3: End-to-end vision-based autonomous driving via spatial-temporal feature learning. In *ECCV*, pages 533–549. Springer, 2022. 1, 2
- [22] Xuemin Hu, Shen Li, Tingyu Huang, Bo Tang, Rouxing Huai, and Long Chen. How simulation helps autonomous driving: A survey of sim2real, digital twins, and parallel intelligence. *IEEE T-IV*, 2023. 2
- [23] Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqi Chai, Senyao Du, Tianwei Lin, Wenhai Wang, et al. Planning-oriented autonomous driving. In *CVPR*, pages 17853–17862, 2023. 1, 2, 3, 5, 6, 7, 8
- [24] Bernhard Jaeger, Kashyap Chitta, and Andreas Geiger. Hidden biases of end-to-end driving models. In *Proc. of the IEEE International Conf. on Computer Vision (ICCV)*, 2023. 12
- [25] Bernhard Jaeger, Kashyap Chitta, and Andreas Geiger. Hidden biases of end-to-end driving models. In *ICCV*, pages 8240–8249, 2023. 1, 3, 4
- [26] Xiaosong Jia, Yulu Gao, Li Chen, Junchi Yan, Patrick Langechuan Liu, and Hongyang Li. Driveadapter: Breaking the coupling barrier of perception and planning in end-to-end autonomous driving. In *ICCV*, pages 7953–7963, 2023. 2, 3, 6, 7, 8
- [27] Xiaosong Jia, Penghao Wu, Li Chen, Jiangwei Xie, Conghui He, Junchi Yan, and Hongyang Li. Think twice before driving: Towards scalable decoders for end-to-end autonomous driving. In *CVPR*, pages 21983–21994, 2023. 2, 6, 7
- [28] Xiaosong Jia, Zhenjie Yang, Qifeng Li, Zhiyuan Zhang, and Junchi Yan. Bench2drive: Towards multi-ability benchmarking of closed-loop end-to-end autonomous driving. In *NeurIPS*, 2024. 2, 3, 5, 11, 12

- [29] Bo Jiang, Shaoyu Chen, Qing Xu, Bencheng Liao, Jiajie Chen, Helong Zhou, Qian Zhang, Wenyu Liu, Chang Huang, and Xinggang Wang. Vad: Vectorized scene representation for efficient autonomous driving. In *ICCV*, pages 8340–8350, 2023. 1, 2, 3, 5, 6, 7, 8
- [30] Chiyu Jiang, Andre Cornman, Cheolho Park, Benjamin Sapp, Yin Zhou, Dragomir Anguelov, et al. Motiondiffuser: Controllable multi-agent motion prediction using diffusion. In *CVPR*, pages 9644–9653, 2023. 3
- [31] Abhishek Kadian, Joanne Truong, Aaron Gokaslan, Alexander Clegg, Erik Wijmans, Stefan Lee, Manolis Savva, Sonia Chernova, and Dhruv Batra. Sim2real predictivity: Does evaluation in simulation predict real-world performance? *IEEE Robotics and Automation Letters*, 5(4):6670–6677, 2020. 2
- [32] Qifeng Li, Xiaosong Jia, Shaobo Wang, and Junchi Yan. Think2drive: Efficient reinforcement learning by thinking in latent world model for quasi-realistic autonomous driving (in carla-v2). In *ECCV*, 2024. 2, 3, 4, 5, 6, 11
- [33] Zhenxin Li, Kailin Li, Shihao Wang, Shiyi Lan, Zhiding Yu, Yishen Ji, Zhiqi Li, Ziyue Zhu, Jan Kautz, Zuxuan Wu, et al. Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation. *arXiv preprint arXiv:2406.06978*, 2024. 1, 2, 3, 6, 7, 11, 12
- [34] Zhiqi Li, Zhiding Yu, Shiyi Lan, Jiahao Li, Jan Kautz, Tong Lu, and Jose M Alvarez. Is ego status all you need for open-loop end-to-end autonomous driving? In *CVPR*, pages 14864–14873, 2024. 2, 3
- [35] Bencheng Liao, Shaoyu Chen, Haoran Yin, Bo Jiang, Cheng Wang, Sixu Yan, Xinbang Zhang, Xiangyu Li, Ying Zhang, Qian Zhang, et al. Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving. *arXiv preprint arXiv:2411.15139*, 2024. 7, 12
- [36] T Lin. Focal loss for dense object detection. *arXiv preprint arXiv:1708.02002*, 2017. 7, 11
- [37] I Loshchilov. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017. 5
- [38] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016. 5
- [39] Tung Phan-Minh, Elena Corina Grigore, Freddy A Boulton, Oscar Beijbom, and Eric M Wolff. Covernet: Multimodal behavior prediction using trajectory sets. In *CVPR*, pages 14074–14083, 2020. 3
- [40] Jonah Philion and Sanja Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d. In *ECCV*, pages 194–210. Springer, 2020. 3
- [41] Philip Polack, Florent Althé, Brigitte d’Andréa Novel, and Arnaud de La Fortelle. The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles? In *IEEE T-IV*, pages 812–818. IEEE, 2017. 4, 5
- [42] Rajesh Rajamani. *Vehicle dynamics and control*. Springer Science & Business Media, 2011. 4, 5
- [43] Katrin Renz, Long Chen, Ana-Maria Marcu, Jan Hünemann, Benoit Hanotte, Alice Karnsund, Jamie Shotton, Elahe Arani, and Oleg Sinavski. Carllava: Vision language models for camera-only closed-loop driving. *arXiv preprint arXiv:2406.10165*, 2024. 1, 3
- [44] Katrin Renz, Long Chen, Elahe Arani, and Oleg Sinavski. Simlingo: Vision-only closed-loop autonomous driving with language-action alignment. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 11993–12003, 2025. 12
- [45] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv:2010.02502*, 2020. 11
- [46] Yuval Tassa, Nicolas Mansard, and Emo Todorov. Control-limited differential dynamic programming. In *ICRA*, pages 1168–1175. IEEE, 2014. 4
- [47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 30, 2017. 3
- [48] Wenhao Wang, Jiangwei Xie, ChuanYang Hu, Haoming Zou, Jianan Fan, Wenwen Tong, Yang Wen, Silei Wu, Hanming Deng, Zhiqi Li, et al. Drivemlm: Aligning multi-modal large language models with behavioral planning states for autonomous driving. *arXiv preprint arXiv:2312.09245*, 2023. 1, 3
- [49] Xinshuo Weng, Boris Ivanovic, Yan Wang, Yue Wang, and Marco Pavone. Para-drive: Parallelized architecture for real-time autonomous driving. In *CVPR*, pages 15449–15458, 2024. 1, 2, 3
- [50] Theodor Westny, Björn Olofsson, and Erik Frisk. Diffusion-based environment-aware trajectory prediction. *arXiv preprint arXiv:2403.11643*, 2024. 3
- [51] Penghao Wu, Xiaosong Jia, Li Chen, Junchi Yan, Hongyang Li, and Yu Qiao. Trajectory-guided control prediction for end-to-end autonomous driving: A simple yet strong baseline. *NeurIPS*, 35:6119–6132, 2022. 1, 2, 3, 4, 5, 6, 7
- [52] Zhou Xian, Nikolaos Gkanatsios, Theophile Gervet, Tsung-Wei Ke, and Katerina Fragkiadaki. Chaineddiffuser: Unifying trajectory diffusion and keypose prediction for robotic manipulation. In *CoRL*, 2023. 3
- [53] Brian Yang, Huangyuan Su, Nikolaos Gkanatsios, Tsung-Wei Ke, Ayush Jain, Jeff Schneider, and Katerina Fragkiadaki. Diffusion-es: Gradient-free planning with diffusion for autonomous driving and zero-shot instruction following. *arXiv preprint arXiv:2402.06559*, 2024. 3
- [54] Chengran Yuan, Zhanqi Zhang, Jiawei Sun, Shuo Sun, Zefan Huang, Christina Dao Wen Lee, Dongen Li, Yuhang Han, Anthony Wong, Keng Peng Tee, et al. Drama: An efficient end-to-end motion planner for autonomous driving with mamba. *arXiv preprint arXiv:2408.03601*, 2024. 12
- [55] Zhejun Zhang, Alexander Liniger, Dengxin Dai, Fisher Yu, and Luc Van Gool. End-to-end urban driving by imitating a reinforcement learning coach. In *ICCV*, pages 15222–15232, 2021. 2, 3

Hydra-NeXt: Robust Closed-Loop Driving with Open-Loop Training

补充材料

7. Hydra-NeXt 训练细节

本节详细介绍了 Hydra-NeXt 的训练细节，包括损失函数和扩散策略的公式 π_{dp} 。Hydra-NeXt 中使用的三种策略 (π_{traj} , π_{ctrl} 和 π_{dp} 均在 Bench2Drive 数据集 [28] 上端到端同时训练，该数据集由基于强化学习的专家 Think2Drive[32] 收集。未使用来自不同专家（如 PDMLite[2]）的外部数据。

7.1. π_{traj} ：轨迹解码器

π_{traj} 的损失与 Hydra-MDP[33] 一致，包含两项：模仿损失 $\mathcal{L}_{im}$ 和知识蒸馏损失 $\mathcal{L}_{kd}$ 。我们将 π_{traj} 中使用的 k 个轨迹锚点记为 $\{T_i\}_{i=1}^k$ 。模仿损失基于专家轨迹 $\hat{T}$ 和每个轨迹锚点的预测模仿分数 $\{\hat{S}_i^{im}\}_{i=1}^k$ 计算为交叉熵：

$$y_i = \frac{e^{-(\hat{T}-T_i)^2}}{\sum_{j=1}^k e^{-(\hat{T}-T_j)^2}} \quad (1)$$

$$\mathcal{L}_{im} = -\sum_{i=1}^k y_i \log(\hat{S}_i^{im}).$$

y_i 衡量专家轨迹 $\hat{T}$ 之间的相似度 and each trajectory anchor T_i . The knowledge distillation loss aims to learn open-loop objectives via binary cross-entropy between predicted metric scores $\{\hat{S}_i^m\}_{i=1}^k$ and the ground-truth metric scores $\{S_i^m\}_{i=1}^k$:

$$\mathcal{L}_{kd} = -\sum_{m,i} \hat{S}_i^m \log S_i^m + (1 - \hat{S}_i^m) \log(1 - S_i^m) \quad (2)$$

where $m \in \{COL, SLK, EP\}$.

COL, SLK, EP 对应于第 3 节中定义的碰撞、软车道保持和自行车进度度量。真实度量分数为二值，由特权信息和每个轨迹锚点推导得出。 π_{traj} 的总体损失公式为：

$$\mathcal{L}_{traj} = \mathcal{L}_{im} + \mathcal{L}_{kd}. \quad (3)$$

7.2. π_{ctrl} ：控制解码器

π_{ctrl} 的损失为每个控制信号（即刹车、油门和转向）在 t_{ctrl} 个时间步上计算基于分类的损失。为简化，控制信号元组 $C = (brake, throttle, steer)$ 缩写为 (b, th, s) ，专家演示记为 $(\hat{b}, \hat{th}, \hat{s})$ 。具体损失函数公式如下：

$$\begin{aligned} \mathcal{L}_{brake} &= \sum_{t=1}^{t_{ctrl}} Focal(b^t, \hat{b}^t) \\ \mathcal{L}_{throttle} &= \sum_{t=1}^{t_{ctrl}} CE(th^t, \hat{th}^t) \\ \mathcal{L}_{steer} &= \sum_{t=1}^{t_{ctrl}} CE(s^t, \hat{s}^t) \end{aligned} \quad (4)$$

$$\mathcal{L}_{ctrl} = \mathcal{L}_{brake} + \mathcal{L}_{throttle} + \mathcal{L}_{steer},$$

其中 $Focal$ 表示焦点损失 [36]， CE 为交叉熵损失。我们通过实验发现，对油门和转向预测应用焦点损失会导致性能下降，可能由于它们在数据集中分布更均衡。

7.3. π_{dp} 扩散策略

π_{dp} 基于在连续数据上训练的标准扩散策略 [8]。给定跨 t_{dp} 帧的专家控制信号 $(\hat{C}^1, \dots, \hat{C}^{t_{dp}})$ ， π_{dp} 被训练用于预测添加到专家控制信号中的噪声 ε^j ，其中 j 为去噪迭代次数。采用均方误差损失进行噪声预测：

$$\mathcal{L}_{dp} = MSE(\varepsilon^j, \pi_{dp}((\hat{C}^1, \dots, \hat{C}^{t_{dp}}) + \varepsilon^j, j)). \quad (5)$$

最终，Hydra-NeXt 的整体损失变为

$$\mathcal{L} = \mathcal{L}_{traj} + \mathcal{L}_{ctrl} + \mathcal{L}_{dp}. \quad (6)$$

8. 各策略的性能表现

我们在 Bench2Drive 基准上对各策略的性能进行了实验。如表 10 所示，与完整版 Hydra-NeXt 相比，仅使用 π_{ctrl} 会导致性能严重下降（驾驶得分 -16.56，成功率 -29.33）。此外，当控制候选遵循 π_{traj} 预测的轨迹而非随机选择时， π_{dp} 取得了更好的结果（驾驶得分 +5.36，成功率 +4.28），这凸显了轨迹引导的重要性。最后，与基线 Hydra-MDP (π_{traj}) 相比，完整版 Hydra-NeXt 实现了显著提升（驾驶得分 +13.09，成功率 +17.47）。

π_{traj}	π_{ctrl}	π_{dp}	Driving Score $\uparrow$	Success Rate $\uparrow$
✓			52.80	30.73
	✓		49.33	18.87
		Rand.	51.18	27.98
		Traj.	56.54	32.26
✓	✓	✓	65.89	48.20

表 10. 各策略在 Bench2Drive 上的性能表现。

9. 高效扩散策略 π_{dp}

我们发现，在不牺牲过多性能的前提下，可以大幅提升 π_{dp} 的效率。通过用 DDIM 调度器 [45] 替换 DDPM，可将延迟降低约 53%，去噪步数从 100 步减少到 20 步。此外，将此修改与 Flash 注意力相结合，我们实现了更低的延迟

Method	Perception Network	Grid Search	NC $\uparrow$	DAC $\uparrow$	TTC $\uparrow$	EP $\uparrow$	C $\uparrow$	PDMS $\uparrow$
Transfuser [9]	Transfuser [9]	-	97.7	92.8	92.8	79.2	100	84.0
DRAMA [54]	Transfuser [9]*	-	98.0	93.1	94.8	80.1	100	85.5
Hydra-MDP [33]	Transfuser [9]	$\times$	97.9	91.7	92.9	77.6	100	83.0
Hydra-MDP [33]	Transfuser [9]	$\checkmark$	98.3	96.0	94.6	78.7	100	86.5
DiffusionDrive [35]	Transfuser [9]	-	98.2	96.2	94.7	82.2	100	88.1
Hydra-NeXt	Transfuser [9]	$\times$	98.4	95.9	94.8	80.6	100	87.2
Hydra-NeXt	Transfuser [9]	$\checkmark$	98.1	97.7	94.6	81.8	100	88.6

表 9. 端到端自动驾驶方法在 NAVSIM 上的性能。以上所有方法均使用 Transfuser [9] 与 ResNet34 [18] 作为感知骨干网络。*DRAMA [54] 使用 Mamba [10] 进行多模态交互。Hydra-MDP [33] 使用网格搜索获得用于加权不同预测度量分数的最优超参数。

Method	Latency (ms) $\downarrow$	D.S. $\uparrow$	S.R. $\uparrow$
VAD	224.3	39.42	10.00
Hydra-NeXt (w/o π_{dp})	250.6	60.40	48.10
Hydra-NeXt* (DDPM)	528.3	65.89	48.20
Hydra-NeXt* (DDIM)	243.9	64.87	46.63

表 11. 不同扩散调度器的效率。*表示使用 Flash-attention [11]。

与 VAD 相当，同时保持强大的闭环性能，驾驶分数仅下降 1%。因此，我们得出结论， π_{dp} 在正确的设计选择下可以既有效又高效。

10. NAVSIM 上的实现

我们在 NAVSIM 上实现 Hydra-NeXt 时，遵循 Hydra-MDP [33]，使用了不同的感知网络 Transfuser [9]。Transfuser [9] 具有两个骨干网络，分别用于相机和激光雷达特征提取，一个鸟瞰图分割头，一个三维目标检测头，以及用于多模态特征交互的变换器层。这种设置有助于与 Hydra-MDP 和 DiffusionDrive [35] 等基线进行公平比较。对于 π_{ctrl} 和 π_{dp} ，我们采用了与 Bench2Drive 上相同的变换器架构来预测加速度和转向速率，因为 NAVSIM [13] 不使用类似 CARLA [1, 15] 的控制信号，仅评估轨迹。因此，这些辅助预测仅作为额外的学习目标。结果，当在 π_{traj} 中采用不同度量分数间的网格搜索技巧 [33] 时，Hydra-NeXt 超越了最先进的规划器 DiffusionDrive，领先 0.5 PDMS（见表 9）。

11. CARLA-Garage 上的实现

我们提供了在 CARLA-Garage 数据集 [24] 上的额外结果，该数据集由 PDM-Lite 专家 [2] 收集。与 Bench2Drive 数据集 [28] 相比，该数据集的特点是驾驶平稳，无抖动行为。我们进行了实验

Method	Expert	D.S. $\uparrow$	S.R. $\uparrow$
Hydra-NeXt	Think2Drive	73.86	50.00
TF++† [24]	PDM-Lite	84.21	67.27
SimLingo [44]	PDM-Lite	85.94	66.82
Hydra-NeXt*†	PDM-Lite	86.00	68.18

表 12. 使用不同专家训练的模型性能。†表示使用不同种子训练的三个模型的集成。

在 TF++ 框架 [24] 中的 π_{traj} 与 π_{dp} 。具体而言，我们将 TF++ 的纵向控制头替换为 π_{traj} （模仿与碰撞头），并将 π_{dp} 的输出与纵向和横向控制相融合。随后，我们集成三个使用不同随机种子训练的模型。该变体 Hydra-NeXt* 达到了 86% 的驾驶得分，超越了现有方法。

12. 可视化结果

图 5 展示了交互场景（汇入、超车、让行）中的更多可视化结果。扩散策略 π_{dp} 能够捕捉多种规划模式，例如跟随其他智能体或超越它们（见图中的第二行和第四行）。

13. 局限性。

尽管与端到端方法相比，Hydra-NeXt 在闭环驾驶性能上表现突出，但仍落后于使用特权输入的基于强化学习的专家。扩散策略的运行时效率也值得优化。我们期望这些问题能在未来的研究中得到解决。

●●● 来自 π_{dp} 的提议 来自 π_{traj} 的轨迹 最近提议

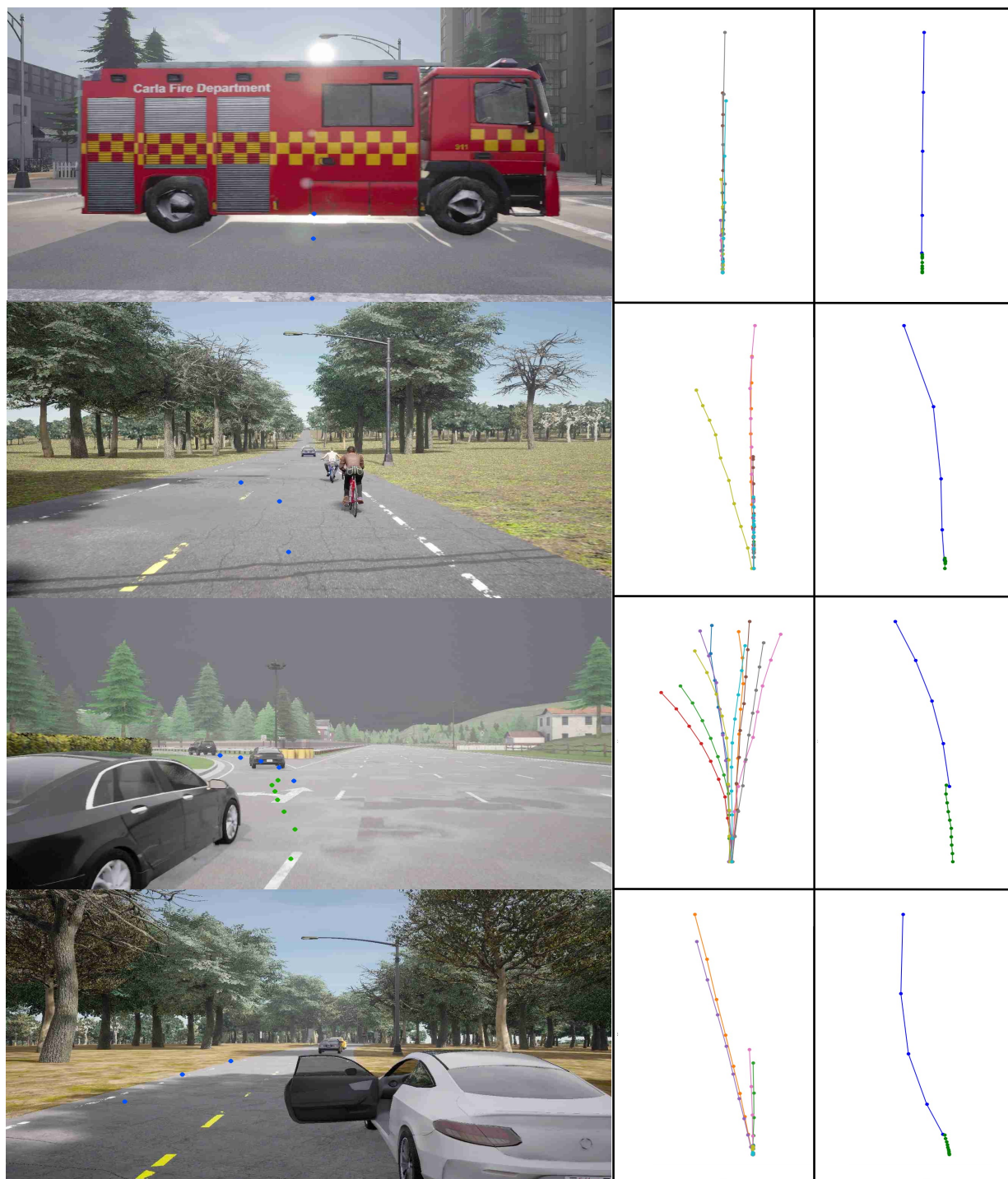


图 5. 轨迹细化的更多可视化。图中展示了前视图像、由控制提议转换而来的轨迹，以及所选的最接近预测轨迹的提议。